

MODULE 24

Transformers and ChatGPT Evolution

Lectures Covered

- Lecture 1 — How LLMs Work Introduction
- Lecture 2 — TikTokenizer and Tokenization in Transformers
- Lecture 3 — OpenAI and HuggingFace for Embeddings
- Lecture 4 — Positional Encoding and Self Attention
- Lecture 5 — Multi-Head Attention and Encoder Other Layers
- Lecture 6 — Decoder Working and HuggingFace Predictions
- Lecture 7 — History of LLMs
- Lecture 8 — Encoder-Decoder for Seq-to-Seq Data
- Lecture 9 — Attention Mechanism
- Lecture 10 — Transformers in the Lifecycle
- Lecture 11 — Transfer Learning with Transformers
- Lecture 12 — Transfer Learning with Language Modelling Task
- Lecture 13 — LLMs from OpenAI and Google
- Lecture 14 — ChatGPT from LLMs

Lecture 1: How LLMs Work — Introduction

A Large Language Model (LLM) is a deep neural network trained on massive text corpora to predict the next token. The same autoregressive objective powers GPT-2, GPT-4, LLaMA, and Claude — just at different scales.

LLM Pipeline at a Glance

Input to Output

1. Raw text -> Tokenizer -> integer token IDs
2. Token IDs -> Embedding layer -> dense vectors
3. Dense vectors -> Transformer blocks (attention + FFN) -> contextual representations
4. Final representation -> Linear + Softmax -> probability distribution over vocabulary
5. Sample/argmax from distribution -> next token -> repeat (autoregressive generation)

Key Properties of LLMs

- Scale: billions of parameters trained on trillions of tokens — scale is the primary driver of capability.
- Emergent abilities: reasoning, code generation, and in-context learning appear only beyond a certain size.
- Few-shot learning: LLMs can perform new tasks from just 2-3 examples in the prompt — no gradient updates.
- The Transformer architecture (Vaswani et al., 2017) enables all modern LLMs — RNNs proved too slow and limited.

Component	Role in LLM
Tokenizer	Splits text into subword tokens; maps to integer IDs
Embedding	Converts token IDs to dense continuous vectors
Transformer	Processes sequence using self-attention; builds contextual representations
LM Head	Linear layer mapping final hidden state to vocabulary logits
Sampling	Selects next token from logits (greedy, top-k, nucleus, temperature)

Lecture 2: TikTokenizer and Tokenization in Transformers

Tokenization converts raw text into integer token IDs that the model can process. Modern LLMs use subword tokenization (BPE or WordPiece) — not word-level or character-level.

Byte Pair Encoding (BPE)

- Start with individual characters as vocabulary. Iteratively merge the most frequent adjacent pair.
- Result: common words become single tokens; rare words split into subword pieces.
- GPT models use BPE. BERT uses WordPiece (similar but merges based on likelihood, not frequency).

```
# TikToken — OpenAI's fast BPE tokenizer
import tiktoken

enc = tiktoken.encoding_for_model('gpt-4')

text = 'Transformers changed natural language processing forever.'
tokens = enc.encode(text)
print('Token IDs:', tokens)
print('Token count:', len(tokens))
print('Decoded:', enc.decode(tokens))

# Inspect individual tokens
for tok_id in tokens:
    print(f'{tok_id:6d} -> {repr(enc.decode([tok_id]))}')

# HuggingFace tokenizer
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained('gpt2')
out = tokenizer('Hello world!', return_tensors='pt')
print(out['input_ids'])    # tensor of token IDs
```

Tokenizer Type	Model Vocab Size Method
BPE	GPT-2/3/4, LLaMA 50K-128K Merge most frequent pairs
WordPiece	BERT, DistilBERT 30K Merge by likelihood ratio
SentencePiece	T5, Gemma, LLaMA3 32K-256K Language-independent BPE
Character	Early models Small One token per character

Lecture 3: OpenAI and HuggingFace for Embeddings

Embeddings convert token IDs into dense vectors. In Transformers, embeddings also carry positional information. Both OpenAI API and HuggingFace provide embedding access.

Token + Positional Embeddings

```
# In a Transformer, the input to the first layer is:
# input_embedding = token_embedding + positional_encoding

# OpenAI Embeddings API
from openai import OpenAI
client = OpenAI() # uses OPENAI_API_KEY env var

response = client.embeddings.create(
    model = 'text-embedding-3-small', # 1536 dims
    input = 'The Transformer architecture'
)
vector = response.data[0].embedding # list of 1536 floats
print(f'Embedding dim: {len(vector)}')

# HuggingFace Sentence Transformers
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')
emb = model.encode('The Transformer architecture')
print(f'Shape: {emb.shape}') # (384,)

# Semantic similarity
from sklearn.metrics.pairwise import cosine_similarity
e1 = model.encode('I love machine learning')
e2 = model.encode('I enjoy deep learning')
print(cosine_similarity([e1], [e2])[0][0]) # ~0.85
```

Model	Dims Best For
text-embedding-3-small	1536 Semantic search, RAG, fast retrieval
text-embedding-3-large	3072 High-accuracy similarity tasks
all-MiniLM-L6-v2	384 Lightweight, local, fast deployment
all-mpnet-base-v2	768 High quality, balanced speed/accuracy

Lecture 4: Positional Encoding and Self-Attention

Transformers have no built-in notion of order — unlike RNNs. Positional Encoding injects sequence order. Self-Attention then lets every token directly attend to every other token in $O(1)$ depth.

Positional Encoding

The original Transformer uses sinusoidal encodings added to token embeddings. Modern models (GPT-NeoX, LLaMA) use Rotary Positional Embeddings (RoPE) baked into attention.

```
import numpy as np
```

```
def sinusoidal_PE(seq_len, d_model):
    PE = np.zeros((seq_len, d_model))
    for pos in range(seq_len):
        for i in range(0, d_model, 2):
            PE[pos, i] = np.sin(pos / 10000 ** (i / d_model))
            PE[pos, i+1] = np.cos(pos / 10000 ** (i / d_model))
    return PE

# input_to_transformer = token_embedding + PE
```

Self-Attention — Scaled Dot-Product

Each token creates three vectors: Query (Q), Key (K), Value (V). Attention computes how much each token should attend to every other token.

```
# For a sequence of length T, embedding dim d_model:
# Q, K, V each have shape (T, d_k) where d_k = d_model / num_heads

# Attention(Q, K, V) = softmax( Q * K^T / sqrt(d_k) ) * V
#                      ^--- attention weights (T x T) --- ^--- context

# Step by step:
# 1. scores = Q @ K.T / sqrt(d_k)    shape: (T, T)
# 2. weights = softmax(scores)        each row sums to 1
# 3. output  = weights @ V            shape: (T, d_k)

# Causal mask for decoder (GPT-style):
# Set upper triangle of scores to -inf before softmax
# -> prevents attending to future tokens
```

Term	Role Shape
Query (Q)	What this token is looking for (T, d _k)
Key (K)	What each token offers (T, d _k)
Value (V)	The actual content to aggregate (T, d _k)
Scores	$Q \cdot K^T / \sqrt{d_k}$ — raw attention weights (T, T)
Weights	$\text{softmax}(\text{scores})$ — normalised attention (T, T)
Output	$\text{weights} \cdot V$ — context-aware representation (T, d _k)

Lecture 5: Multi-Head Attention and Encoder Layers

Multi-Head Attention (MHA) runs h independent attention heads in parallel, each learning different aspects of the relationships between tokens. The encoder stacks multiple MHA + FFN blocks.

Multi-Head Attention

```
# MultiHead(Q, K, V) = Concat(head_1, ..., head_h) * W_O
# where head_i = Attention(Q*W_Qi, K*W_Ki, V*W_Vi)

# Each head has its own W_Q, W_K, W_V (d_model x d_k)
# d_k = d_model / h (split evenly across heads)

# Example: d_model=512, h=8 -> d_k=64 per head
# 8 heads in parallel -> each attends to different patterns
# Head 1 might track syntax, Head 3 might track coreference...
```

Full Encoder Block

Encoder block (repeated N times)

1. Multi-Head Self-Attention -> each token attends to ALL tokens

2. Add & Norm (residual connection + LayerNorm)
3. Feed-Forward Network (FFN): two linear layers with ReLU/GELU
 $\text{FFN}(x) = \max(0, x \cdot W_1 + b_1) \cdot W_2 + b_2$ ($d_{\text{model}} \rightarrow 4 \cdot d_{\text{model}} \rightarrow d_{\text{model}}$)
4. Add & Norm (second residual connection)

Component	Purpose Key Detail
Multi-Head Attention	Captures relationships between all token pairs h heads in parallel
Residual Connection	$x + \text{sublayer}(x)$ — prevents vanishing gradients in deep stacks
Layer Normalisation	Normalises across feature dim — stabilises training
Feed-Forward Network	Applies non-linearity token-wise — 4x expansion then back

Model	d_{model} Heads Layers Parameters
BERT-base	768 12 12 110M
BERT-large	1024 16 24 340M
GPT-2 small	768 12 12 117M
GPT-3	12288 96 96 175B

Lecture 6: Decoder Working and HuggingFace Predictions

The Transformer decoder generates output tokens autoregressively. It has three sub-layers per block: Masked Self-Attention, Cross-Attention (attending to encoder output), and FFN.

Decoder Block

Decoder block (3 sub-layers)

1. Masked Self-Attention — attend only to past positions (causal mask)
2. Cross-Attention (Encoder-Decoder Attention) — Q from decoder, K/V from encoder
3. Feed-Forward Network + Add & Norm at each step

```
# HuggingFace — text generation with GPT-2
from transformers import pipeline

gen = pipeline('text-generation', model='gpt2')
out = gen('The future of artificial intelligence is',
          max_new_tokens=50, num_return_sequences=1)
print(out[0]['generated_text'])

# Using AutoModel for custom inference
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

tokenizer = AutoTokenizer.from_pretrained('gpt2')
model = AutoModelForCausalLM.from_pretrained('gpt2')

inputs = tokenizer('To be or not to be', return_tensors='pt')
outputs = model.generate(**inputs, max_new_tokens=20,
                        do_sample=True, temperature=0.8)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

Generation Parameter	Effect
temperature	< 1 = conservative, > 1 = more random/creative
top_k	Sample only from top K tokens at each step
top_p (nucleus)	Sample from smallest set of tokens with cumulative prob $\geq p$
do_sample	True = sampling, False = greedy decoding

Generation Parameter	Effect
num_beams	Beam search — tracks top N sequences simultaneously

Lecture 7: History of LLMs

The evolution from n-gram language models to GPT-4 took ~30 years. Each breakthrough solved a specific limitation of the previous approach.

Year	Model / Paper	Key Contribution
2013	Word2Vec (Mikolov)	Static word embeddings; semantic arithmetic (king-man+woman=queen)
2014	GRU (Cho et al.)	Simpler gated RNN; seq2seq architecture for machine translation
2015	Attention (Bahdanau)	Decoder attends to all encoder states — solved seq2seq bottleneck
2017	Transformer (Vaswani)	'Attention Is All You Need' — replaced RNNs entirely
2018	BERT (Google)	Bidirectional encoder; pre-train on MLM + NSP; fine-tune downstream
2018	GPT-1 (OpenAI)	Decoder-only; pre-train on next-token prediction; transfer learning
2019	GPT-2 (OpenAI)	1.5B params; zero-shot task transfer; too dangerous to release (initially)
2020	GPT-3 (OpenAI)	175B params; few-shot learning; in-context learning emergent ability
2022	ChatGPT (OpenAI)	GPT-3.5 + RLHF — aligned to follow instructions conversationally
2023	GPT-4 (OpenAI)	Multimodal, 1.8T MoE params; top human-level on many benchmarks
2023	LLaMA / LLaMA 2 (Meta)	Open-weight competitive models; enabled open-source LLM ecosystem
2024	Gemini (Google)	Multimodal from ground up; 1M context window (1.5 Pro)

Lecture 8: Encoder-Decoder for Seq-to-Seq Data

The original Transformer was designed as an Encoder-Decoder for machine translation. This architecture is ideal when input and output are sequences of different lengths and different domains.

Architecture Overview

Component	Role Examples
Encoder	Reads full input sequence; produces contextual representations BERT, RoBERTa
Decoder	Generates output autoregressively; attends to encoder via cross-attention GPT
Encoder-Decoder	Full architecture: translate / summarise / answer T5, BART, mT5

```
# T5 - Text-to-Text Transfer Transformer
# Every NLP task framed as: input text -> output text
#
# Translation:    'translate English to French: Hello world'
# Summarisation: 'summarize: The article states that...'
# Q&A:           'question: What is the capital? context: Paris is...'
```

```
from transformers import T5ForConditionalGeneration, T5Tokenizer
```

```
tokenizer = T5Tokenizer.from_pretrained('t5-small')
model     = T5ForConditionalGeneration.from_pretrained('t5-small')
```

```
input_text = 'summarize: Transformers use self-attention to process sequences.'
inputs     = tokenizer(input_text, return_tensors='pt')
outputs    = model.generate(**inputs, max_new_tokens=50)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

Task	Encoder-Decoder Model Why enc-dec?
Machine Translation	Helsinki-NLP/opus-mt Input/output differ in length and language
Summarisation	BART, PEGASUS Long input compressed to short output

Task	Encoder-Decoder Model Why enc-dec?
Question Answering	T5, BART Context + question -> answer (different length)
Code Generation	CodeT5 Natural language -> code (different domain)

Lecture 9: Attention Mechanism

The Attention mechanism is the core innovation that enabled Transformers. It allows every token to directly access information from any other token in $O(1)$ steps, eliminating the vanishing gradient problem of RNNs.

Three Types of Attention in Transformers

Type	Where Used Q, K, V Sources Purpose
Self-Attention (encoder)	Encoder blocks Q=K=V from encoder Each token attends to all input tokens
Masked Self-Attention (decoder)	Decoder blocks Q=K=V from decoder Each token attends to only past tokens
Cross-Attention	Decoder blocks Q from decoder, K/V from encoder Decoder reads encoder output

Intuition: Why Attention Solves Long-Range Dependencies

Gradient path comparison

RNN: gradient from token 50 back to token 1 passes through 50 multiplications -> vanishes.

Attention: token 50 directly attends to token 1 via $Q \cdot K^T$ -> gradient path length = 1.

This is why Transformers learn long-range dependencies that RNNs cannot.

```
import torch, torch.nn.functional as F, math

def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.size(-1)
    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))
    weights = F.softmax(scores, dim=-1)
    return torch.matmul(weights, V), weights

# weights shape: (batch, heads, T, T) - T x T attention map
# output shape: (batch, heads, T, d_k)
```

Lecture 10: Transformers in the Lifecycle

Transformers appear at every stage of the modern NLP/AI lifecycle — from pre-training to fine-tuning to deployment. Understanding where each model type fits prevents common architecture mismatches.

Lifecycle Stage	Transformer Role Typical Model
Pre-training	Train on massive unlabelled corpus (MLM or CLM) BERT, GPT, T5, LLaMA
Fine-tuning	Adapt pre-trained model to specific task with labelled data BERT -> classifier
Instruction tuning	Fine-tune on (instruction, response) pairs Flan-T5, InstructGPT
RLHF	Align model output with human preferences via reward model ChatGPT, Claude
Inference / Deployment	Serve fine-tuned model via API or locally vLLM, TGI, Ollama
RAG	Augment LLM with retrieved documents at inference time GPT + vector DB

Encoder vs Decoder vs Encoder-Decoder

Architecture	Training Objective Best For Examples
Encoder-only	Masked LM (predict [MASK]) Understanding, classification, NER BERT, RoBERTa

Architecture	Training Objective Best For Examples
Decoder-only	Causal LM (predict next) Generation, chat, code GPT, LLaMA, Gemma
Encoder-Decoder	Seq2Seq (input->output) Translation, summarisation T5, BART

Lecture 11: Transfer Learning with Transformers

Transfer learning with Transformers follows the pre-train then fine-tune paradigm. A pre-trained model's weights encode rich language understanding — fine-tuning adapts them to a specific downstream task in minutes.

Fine-Tuning BERT for Classification

```
from transformers import (BertForSequenceClassification,
                          BertTokenizer, Trainer, TrainingArguments)
from datasets import load_dataset

# Load pre-trained BERT
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased', num_labels=2)

# Tokenise dataset
dataset = load_dataset('imdb')
def tokenise(batch):
    return tokenizer(batch['text'], truncation=True, padding='max_length',
                     max_length=512)
dataset = dataset.map(tokenise, batched=True)

# Fine-tune
args = TrainingArguments(output_dir='./bert-imdb', num_train_epochs=3,
                        per_device_train_batch_size=16,
                        evaluation_strategy='epoch')
trainer = Trainer(model=model, args=args,
                  train_dataset=dataset['train'],
                  eval_dataset=dataset['test'])
trainer.train()
```

Fine-Tuning Strategy	When to Use What Trains
Full fine-tuning	Large labelled dataset, different domain All layers
Feature extraction	Small dataset, similar domain Only classification head
LoRA / QLoRA	LLMs, any dataset size Low-rank adapter matrices only (~0.1% params)
Prompt tuning	Very small data, frozen model Soft prompt embeddings only

Lecture 12: Transfer Learning with Language Modelling Task

Language Modelling (LM) is the universal pre-training task for Transformers — predicting masked tokens (MLM for BERT) or next tokens (CLM for GPT). Fine-tuning on domain-specific text adapts the model before task-specific tuning.

Causal Language Modelling (CLM) — GPT-style

```
from transformers import AutoModelForCausalLM, AutoTokenizer, Trainer
from transformers import TrainingArguments, DataCollatorForLanguageModeling
from datasets import load_dataset

model_name = 'gpt2'
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token

# Domain-specific corpus (e.g. medical notes)
dataset = load_dataset('text', data_files={'train': 'medical_notes.txt'})
def tokenise(batch):
    return tokenizer(batch['text'], truncation=True,
```

```

        max_length=512, return_special_tokens_mask=True)
dataset = dataset.map(tokenise, batched=True, remove_columns=['text'])

# CLM data collator - labels = shifted input_ids
collator = DataCollatorForLanguageModeling(tokenizer, mlm=False)

args = TrainingArguments('./gpt2-medical', num_train_epochs=3,
                          per_device_train_batch_size=8)
trainer = Trainer(model=model, args=args,
                  train_dataset=dataset['train'],
                  data_collator=collator)

trainer.train()

```

LM Task	Objective Model Type Use Case
Masked LM (MLM)	Predict [MASK] tokens Encoder BERT pre-training
Causal LM (CLM)	Predict next token Decoder GPT pre-training, generation
Seq2Seq LM	Reconstruct corrupted span Enc-Dec T5, BART pre-training
Domain-adaptive LM	CLM on domain text before task fine-tune Any Medical/legal LLMs

Lecture 13: LLMs from OpenAI and Google

OpenAI Model Family

Model	Params Context Key Capability
GPT-3	175B 4K tokens First truly few-shot LLM; basis for ChatGPT
GPT-3.5 Turbo	~175B 16K Fast, cost-effective; ChatGPT default (2022-23)
GPT-4	~1.8T MoE 128K Multimodal vision + text; human-level on many benchmarks
GPT-4o	Undisclosed 128K Omni: text+audio+vision natively; faster than GPT-4
o1 / o3	Undisclosed 200K Extended chain-of-thought reasoning before answering

Google Model Family

Model	Params Context Key Capability
PaLM 2	340B 32K Multilingual, coding, reasoning; powers Bard
Gemini 1.0 Ultra	Undisclosed 32K Multimodal; claimed to surpass GPT-4 on MMLU
Gemini 1.5 Pro	Undisclosed 1M tokens 1M context window; video + audio + text
Gemma 2	2B/9B/27B 8K Lightweight open-weight; runs on consumer GPU

Accessing Models via API

```

# OpenAI API
from openai import OpenAI
client = OpenAI()
response = client.chat.completions.create(
    model = 'gpt-4o',
    messages = [{'role': 'user', 'content': 'Explain self-attention in 3 sentences.'}]
)
print(response.choices[0].message.content)

# Google Gemini API
import google.generativeai as genai
genai.configure(api_key='YOUR_GOOGLE_API_KEY')
model = genai.GenerativeModel('gemini-1.5-pro')
response = model.generate_content('Explain self-attention in 3 sentences.')
print(response.text)

```

Lecture 14: ChatGPT from LLMs

ChatGPT is GPT-3.5/GPT-4 fine-tuned with Reinforcement Learning from Human Feedback (RLHF) to follow instructions, refuse harmful requests, and maintain multi-turn conversation coherence.

From GPT to ChatGPT — Three Stages

Stage	Process Goal
1. Supervised Fine-Tuning (SFT)	Fine-tune GPT-3 on (prompt, ideal response) pairs written by human labellers Model learns instruction-following format
2. Reward Model Training	Train a separate model to score responses by quality using human preference rankings Reward model learns what humans prefer
3. RLHF (PPO)	Fine-tune SFT model using PPO reinforcement learning; reward signal from reward model Policy optimised for human preference

Why RLHF matters

Pre-trained GPT-3 completes text — it does not follow instructions.

SFT alone teaches format but not alignment to human values.

RLHF teaches the model to: be helpful, be harmless, be honest (HHH).

ChatGPT's refusals, apologies, and safety behaviours all come from RLHF.

ChatGPT Architecture at Inference

```
# ChatGPT uses a system prompt + conversation history
# Every API call includes the full conversation context

messages = [
    {'role': 'system', 'content': 'You are a helpful AI assistant.'},
    {'role': 'user', 'content': 'What is the Transformer architecture?'},
    {'role': 'assistant', 'content': 'The Transformer uses self-attention...'},
    {'role': 'user', 'content': 'How does multi-head attention work?'}
]

# Model sees entire conversation -> maintains coherent multi-turn dialogue
# Context window limits how far back it can see (128K for GPT-4o)
```

ChatGPT Version	Base Model Context Key Upgrade
ChatGPT (Nov 2022)	GPT-3.5 4K First public RLHF chatbot; 100M users in 2 months
GPT-4 (Mar 2023)	GPT-4 8K/32K Multimodal; bar exam top 10%; major accuracy jump
GPT-4o (May 2024)	GPT-4o 128K Omni-modal; faster; free tier; native voice
o1 (Sep 2024)	o1 200K Chain-of-thought reasoning; PhD-level science/math

Key takeaway for practitioners

ChatGPT = pre-trained LLM + SFT + RLHF. The Transformer is just the base.

The capability comes from scale (pre-training) + alignment (RLHF).

You can replicate the process at small scale: fine-tune an open LLM

with SFT using `trl.SFTTrainer`, then add preference optimisation with DPO.

End of Module 24 Notes — Generative AI Course